Leveraging R for Statistical Computing and Clinical Data Analysis

Mahesh Divakaran

Introduction to R

What is R?

- R is a programming language and software environment for statistical computing and graphics.
- It is widely used among statisticians and data miners for developing statistical software and data analysis.
- R provides a wide variety of statistical and graphical techniques, including linear and nonlinear modeling, classical statistical tests, time-series analysis, classification, clustering, and more.
- R is highly extensible, with a large number of packages available for various statistical and graphical techniques.
- R is open-source software, which means it is freely available for anyone to use, modify, and distribute.
- R has a large and active community of users and developers who contribute to its development and provide support through forums, mailing lists, and online resources.

Why use R?

- R is a powerful and flexible tool for data analysis and visualization.
- It has a wide range of statistical and graphical techniques available, making it suitable for a variety of applications.
- R is highly extensible, with a large number of packages available for various statistical and graphical techniques.
- R is open-source software, which means it is freely available for anyone to use, modify and distribute.

Getting Started with R

- To get started with R, you will need to install R and an integrated development environment (IDE) such as RStudio.
- Once you have installed R and RStudio, you can start writing R code in the RStudio console or in R scripts.
- R scripts are plain text files that contain R code and can be saved with the .R extension.
- You can run R code in RStudio by typing it into the console or by running R scripts using the "Run" button or keyboard shortcuts.
- RStudio also provides a variety of tools for data visualization, package management, and debugging.

Basic R Syntax

- R code is typically written in a script or the R console.
- R uses a variety of symbols and keywords to perform different operations.
- Here are some basic R syntax elements:
  - Assignment: `<-` or `=` is used to assign values to variables.
  - Comments: `#` is used to add comments to R code.
  - Functions: Functions are defined using the `function()` keyword.
  - Data Structures: R has several built-in data structures, including vectors, matrices, data frames, and lists.
  - Control Structures: R has several control structures, including `if`, `else`, `for`, and `while` loops.

Example R Code

```r
# Assign a value to a variable
x <- 10
# Create a vector
y <- c(1, 2, 3, 4, 5)
# Define a function
add <- function(a, b) {
  return(a + b)
}
# Use a control structure
if (x > 5) {
  print("x is greater than 5")
} else {
  print("x is less than or equal to 5")
}
```

Why R is Growing in Clinical Analysis

- Regulatory bodies like the **FDA now accept R-based submissions** for clinical trials.
- Many pharmaceutical companies are transitioning to R due to its **cost-effectiveness and flexibility**.
- R offers **extensive statistical and visualization capabilities**, making it suitable for clinical research.
- Open-source nature enables continuous innovation and updates by a global community.

Advantages of R Over SAS

- **Cost-Effective**: R is open-source and free, whereas SAS requires a costly license.
- **Extensive Libraries**: Thousands of packages available for statistical modeling and machine learning.
- **Better Visualization**: R's **ggplot2** package offers superior graphing capabilities compared to SAS ODS.
- **Strong Community Support**: Active global community contributing updates and solutions.

R vs. SAS: Key Differences

Feature	SAS	R
Cost	Licensed (Paid)	Open-source (Free)
Interface	GUI & Code (PROC, DATA Step)	Primarily Code-Based (RStudio, Jupyter)
Data Handling	Structured datasets (SAS datasets)	Flexible (Data frames, tibbles)
Graphing	PROC SGPLOT, ODS Graphics	ggplot2, base R graphics
Extensibility	Limited to SAS procedures	Thousands of packages via CRAN

R's Role in the Future of Data Science

- Increasing adoption in **clinical trials, genomics, and epidemiology**.
- Expanding use in **AI and machine learning** applications.
- More companies are training employees in R due to **its growing demand in the job market**.
- Integration with big data platforms like **Spark and Hadoop**.

Learning R: First Steps

- Install R and RStudio for an interactive development environment.
- Learn base R functions for **data manipulation and statistical analysis**.
- Explore key packages like **dplyr, ggplot2, and tidymodels**.
- Join online communities and forums like **RStudio Community, Stack Overflow, and R-Ladies** for support.

Installing R, RStudio, RTools, and TinyTeX

Step 1: Install R

1. Go to CRAN R Download.
2. Choose your operating system:
   - **Windows**: Click *Download R for Windows* $\rightarrow$ *base* $\rightarrow$ *Download*
   - **Mac**: Click *Download R for macOS* and choose the latest package.
   - **Linux**: Follow the instructions for your distribution.
3. Run the installer and follow the setup instructions.

Step 2: Install RStudio

1. Go to RStudio Download.
2. Download the installer for your operating system.
3. Run the installer and complete the installation.

Step 3: Install RTools (Windows Only)

1. Go to RTools Download.
2. Download the latest RTools installer.
3. Run the installer and follow the instructions.
4. Add RTools to system PATH (if not automatically set).

Step 4: Install TinyTeX (For LaTeX Support)

1. Open RStudio and go to the **Console**.
2. Run the following command:

```
install.packages("tinytex")
tinytex::install_tinytex()
```

3. Restart RStudio to complete the setup.

Step 5: Installing and Loading Packages in R

Installing a Package

1. Open RStudio and go to the **Console**.
2. Use `install.packages("package_name")` to install a package.

```
install.packages("dplyr")
```

Step 5: Installing and Loading Packages in R

Loading a Package

1. Use `library(package_name)` to load a package.

```
library(dplyr)
```

2. If the package is not installed, install it first.

Installation Complete

- **Verify R Installation**: Run `R --version` in the terminal.
- **Verify RStudio**: Open RStudio and check if R is detected.
- **Verify TinyTeX**: Run `tinytex::is_tinytex()` in RStudio.

Syntax Differences: SAS Data Steps vs. R

Basic Syntax Differences: SAS Data Steps vs. Tidyverse in R"

- SAS primarily relies on **DATA steps and PROC SQL** for data manipulation.
- R, especially with the **Tidyverse** package collection, provides a more readable and functional approach.
- This session covers key differences in data handling between SAS and R.

Data Import: SAS vs. R

SAS (Importing a CSV File)

```
PROC IMPORT DATAFILE="data.csv"
    OUT=mydata
    DBMS=CSV
    REPLACE;
    GETNAMES=YES;
RUN;
```

R (Using readr from Tidyverse)

```
library(readr)
mydata <- read_csv("data.csv")
```

- **PROC IMPORT** in SAS → `read_csv()` in R.
- R functions allow more customization and flexibility.

Libname Equivalent in SAS vs R

In SAS, the `libname` statement assigns a shortcut name to a directory or a data library. This makes it easy to reference datasets without specifying full paths each time.

SAS:

```
libname mylib 'C:/data/';
data mydata;
  set mylib.dataset;
run;
```

In R, while there's no direct equivalent to `libname`, there are multiple ways to manage file paths and read data files efficiently.

Reading Flat Files

```
library(readr)
data <- read_csv("C:/data/myfile.csv")
```

Use `read_csv()` to read CSV files directly from a path.

Reading SAS Files in R

```
library(haven)
data <- read_sas("C:/data/myfile.sas7bdat")
```

The `haven` package allows you to import native SAS datasets.

Dynamic Paths with file.path

```r
library(here)
data <- read_csv(here("data", "myfile.csv"))

# Manually building paths
folder <- "C:/data"
file <- "myfile.csv"
path <- file.path(folder, file)
data <- read_csv(path)
```

Using `file.path()` or `here()` helps maintain platform-independent paths and improves project portability.

Project Setup and Folder Management in R

In SAS, you often organize your work using libraries. In R, especially with RStudio, project environments are a better way to maintain reproducibility and folder structure.

Creating an R Project

- In RStudio: Go to File → New Project → New Directory
- Choose an empty project or an R package structure
- This creates an `.Rproj` file which helps track working directory and settings

Updating Folders in a Project

You can manually create folders (e.g., `raw_data`, `scripts`, `output`) or do it programmatically:

```
# Create folders if they don't exist
folders <- c("raw_data", "scripts", "output")
for (f in folders) {
  if (!dir.exists(f)) dir.create(f)
}
```

Input / Put

These functions are used in SAS for reading raw data and printing or logging values. In R, base functions are used to achieve similar results.

SAS:

```
input age $;
put age;
```

R:

```
age <- as.character(23)
print(age)
```

Use `as.character()` to convert to character and `print()` to output values in the console.

Date Handling

SAS uses a d suffix to indicate date literals (e.g., '01JAN2020'd). In R, the as.Date() function is used to convert character strings into Date objects.

Creating Basic Dates

Use `as.Date()` for creating simple date objects.

```
# ISO format (YYYY-MM-DD)
d1 <- as.Date("2025-01-21")
d1
```

SAS-style format: `21JAN2025`

```
d2 <- toupper(format(as.Date("21JAN2025", format = "%d%b%Y"
d2
```

- `%d`: day (2-digit)
- `%b`: abbreviated month (case-insensitive)
- `%Y`: 4-digit year

Creating Date-Time Objects

Use `as.POSIXct()` for date-time combinations.

```
# ISO format with time
dt1 <- as.POSIXct("2025-01-21 10:55", format = "%Y-%m-%d %H
dt1
format(dt1, "%Y-%m-%d %H:%M")  # Returns date-time without
```

SAS-style datetime: `21JAN2025T10:55`

```
dt2 <- as.POSIXct("2025-01-21 10:55", format = "%Y-%m-%d %H
dt2
toupper(format(dt2, "%d%b%YT%H:%M"))
```

Date Formatting Output

Customize how dates appear with `format()`.

```r
d <- as.Date("2025-01-21")

format(d, "%d-%b-%Y")       # 21-Jan-2025
format(d, "%A, %d %B %Y")   # Tuesday, 21 January 2025
```

- `%A`: full weekday name
- `%B`: full month name
- `%m`: numeric month

Extracting Components

```
date <- as.POSIXct("2025-01-21 10:55")

format(date, "%H")  # Hour
format(date, "%M")  # Minute
format(date, "%d")  # Day
format(date, "%b")  # Month (abbreviated)
```

You can also use `lubridate` for more intuitive parsing:

```
library(lubridate)

d <- dmy("21JAN2025")          # from day-month-year
dt <- dmy_hm("21JAN2025 10:55")
```

Tips for Clean Parsing

- Always inspect date formats before converting
- Use `strptime()` to test formats interactively
- Prefer `POSIXct` over `POSIXlt` for storage and plotting

String Operations: scan() and substr()

In SAS, `scan()` is used to split strings and extract specific words. R uses `strsplit()` for similar behavior. `substr()` works in both languages for extracting substrings.

SAS:

```
name = scan(fullname, 1);
part = substr(name, 1, 3);
```

R:

```
strsplit(fullname, " ")[[1]][1]  # Similar to scan
substr(name, 1, 3)                # Same as substr
```

`strsplit()` splits a string into parts, and indexing is used to extract specific elements. `substr()` is used to extract parts of strings.

Filtering Data: SAS vs. R

SAS (Using WHERE in DATA Step)

```
DATA new_data;
    SET mydata;
    WHERE age > 30;
RUN;
```

R (Using dplyr's filter)

```
library(dplyr)
new_data <- mydata %>%
    filter(age > 30)
```

- **WHERE statement (SAS)** → `filter()` function (R).
- Tidyverse allows piping (%>%) for sequential operations.

Creating New Variables: SAS vs. R

SAS (Using DATA Step)

```
DATA new_data;
    SET mydata;
    new_var = old_var * 2;
RUN;
```

R (Using dplyr's mutate)

```
new_data <- mydata %>%
    mutate(new_var = old_var * 2)
```

- **Assignment in DATA step (SAS)** $\rightarrow$ `mutate()` function in R.
- More intuitive in R with chaining operations.

Sorting Data: SAS vs. R

SAS (Using PROC SORT)

```
PROC SORT DATA=mydata OUT=sorted_data;
    BY age;
RUN;
```

R (Using dplyr's arrange)

```
sorted_data <- mydata %>%
    arrange(age)
```

- **PROC SORT (SAS)** $\rightarrow$ `arrange()` function (R).
- R allows sorting by multiple variables using `arrange(var1, desc(var2))`.

Removing Duplicates: SAS vs. R

SAS (Using PROC SORT with NODUPKEY)

```
PROC SORT DATA=mydata NODUPKEY OUT=unique_data;
    BY id;
RUN;
```

R (Using distinct from dplyr)

```
unique_data <- mydata %>%
    distinct(id, .keep_all = TRUE)
```

- **PROC SORT with NODUPKEY (SAS)** $\rightarrow$ `distinct()` function (R).

Summarizing Data: SAS vs. R

SAS (Using PROC MEANS)

```
PROC MEANS DATA=mydata;
    VAR salary;
    CLASS department;
RUN;
```

R (Using dplyr's summarize)

```
summary_data <- mydata %>%
    group_by(department) %>%
    summarize(mean_salary = mean(salary, na.rm = TRUE))
```

- **PROC MEANS (SAS)** → `summarize()` function (R).
- Grouping done using `group_by()` in R.

Merging Data: SAS vs. R

SAS (Using MERGE in DATA Step)

```
DATA merged_data;
    MERGE dataset1 dataset2;
    BY ID;
RUN;
```

R (Using dplyr's join functions)

```
merged_data <- left_join(dataset1, dataset2, by = "ID")
```

- **MERGE statement (SAS)** → `left_join()` or other join functions (R).
- R provides multiple join types (inner, outer, left, right) through `dplyr`.

Conditional Processing: SAS vs. R

SAS (Using IF-THEN)

```
DATA new_data;
    SET mydata;
    IF age > 50 THEN category = "Senior";
    ELSE category = "Junior";
RUN;
```

R (Using dplyr's mutate and if_else)

```
new_data <- mydata %>%
    mutate(category = if_else(age > 50, "Senior", "Junior")
```

- **IF-THEN logic (SAS)** → `if_else()` function (R).

Summary

Operation	SAS (Data Step)	R (Tidyverse)
Import Data	PROC IMPORT	read_csv()
Filter Data	WHERE	filter()
Create Variable	Assignment (=)	mutate()
Sort Data	PROC SORT	arrange()
Remove Duplicates	PROC SORT NODUPKEY	distinct()
Summarize Data	PROC MEANS	summarize()
Merge Data	MERGE	left_join()
Conditional Logic	IF-THEN	if_else()

Working with Data in R

Reading and Writing Data

Reading CSV Files

```r
# Read a CSV file into a data frame
my_data <- read.csv("data.csv")
```

Reading Excel Files

```r
library(readxl)
my_data <- read_excel("data.xlsx", sheet = 1)
```

Reading SAS Files

```r
library(haven)
my_data <- read_sas("data.sas7bdat")
```

Reading Other Formats

```r
# Read a JSON file
library(jsonlite)
my_data <- fromJSON("data.json")
```

Writing Data

Writing CSV Files

```
write.csv(my_data, "output.csv", row.names = FALSE)
```

Writing Excel Files

```
library(writexl)
write_xlsx(my_data, "output.xlsx")
```

Writing SAS Files

```
library(haven)
write_sas(my_data, "output.sas7bdat")
```

Writing JSON Files

```
library(jsonlite)
write_json(my_data, "output.json")
```

Working with Data Frames

Creating a Data Frame

```
data_frame <- data.frame(
  ID = 1:5,
  Name = c("Alice", "Bob", "Charlie", "David", "Eve"),
  Age = c(25, 30, 35, 40, 45)
)
```

Working with Data Frames

Viewing Data

```
head(data_frame)
tail(data_frame)
str(data_frame)
summary(data_frame)
```

Working with Data Frames

Manipulating Data

```r
# Selecting Columns
data_frame[, c("ID", "Name")]

# Filtering Rows
subset(data_frame, Age > 30)

# Adding a New Column
data_frame$Salary <- c(50000, 55000, 60000, 65000, 70000)
```

Working with Data Frames

Removing Missing Values

```
data_frame <- na.omit(data_frame)
```

Summary

- **Read and write various file formats** (CSV, Excel, SAS, JSON, TXT)
- **Create and manipulate data frames**
- **Filter, select, and clean data efficiently**

Ready to move on to data manipulation techniques with `dplyr`?

Data Manipulation

Introduction to Data Manipulation

- Transforming and cleaning data efficiently
- Comparing **Tidyverse (R)** and **PROC SQL (SAS)**
- **Why is data manipulation important?**
  - Ensures data accuracy and consistency
  - Prepares raw data for analysis
  - Facilitates better decision-making
- **Key operations:**
  - **Filtering**: Extracting specific subsets of data
  - **Aggregation**: Summarizing numerical data
  - **Joins**: Merging multiple datasets
  - **Sorting**: Organizing data efficiently
  - **Case Statements**: Conditional processing of data

Filtering Data

SAS (PROC SQL)

```
PROC SQL;
    SELECT * FROM employees
    WHERE salary > 50000;
QUIT;
```

R (Tidyverse)

```
library(dplyr)
employees %>%
  filter(salary > 50000)
```

Filtering Data

Additional filtering operations:

- **Multiple conditions:**
  - SAS: `WHERE salary > 50000 AND department = 'HR';`
  - R: `filter(salary > 50000, department == "HR")`
- **Using OR condition:**
  - SAS: `WHERE salary > 50000 OR department = 'HR';`
  - R: `filter(salary > 50000 | department == "HR")`
- **Using IN condition:**
  - SAS: `WHERE department IN ('HR', 'Finance');`
  - R: `filter(department %in% c("HR", "Finance"))`

Aggregation & Summarization

SAS (PROC SQL)

```
PROC SQL;
   SELECT department, AVG(salary) AS avg_salary, COUNT(*) A
   FROM employees
   GROUP BY department;
QUIT;
```

R (Tidyverse)

```
employees %>%
  group_by(department) %>%
  summarise(
    avg_salary = mean(salary),
    total_employees = n()
  )
```

Additional aggregation operations:

- `SUM(salary) AS total_salary` (SAS) → `summarise(total_salary = sum(salary))` (R)

Sorting Data

SAS (PROC SQL)

```
PROC SQL;
   SELECT * FROM employees
   ORDER BY salary DESC;
QUIT;
```

R (Tidyverse)

```
employees %>%
  arrange(desc(salary))
```

Sorting with multiple columns:

- `ORDER BY department, salary DESC` (SAS) → `arrange(department, desc(salary))` (R)

Joins (Merging Datasets)

SAS (PROC SQL)

```
PROC SQL;
   SELECT a.*, b.department
   FROM employees a
   LEFT JOIN departments b
   ON a.dept_id = b.dept_id;
QUIT;
```

R (Tidyverse)

```
employees %>%
  left_join(departments, by = "dept_id")
```

Other join types:

- **Inner Join:** `inner_join(employees, departments, by = "dept_id")`
- **Right Join:** `right_join(employees, departments, by = "dept_id")`
- **Full Join:** `full_join(employees, departments, by =`

Case Statements & Conditional Processing

SAS (PROC SQL)

```
PROC SQL;
   SELECT name, salary,
   CASE
      WHEN salary > 70000 THEN 'High'
      WHEN salary BETWEEN 40000 AND 70000 THEN 'Medium'
      ELSE 'Low'
   END AS salary_category
   FROM employees;
QUIT;
```

R (Tidyverse)

```
employees %>%
  mutate(salary_category = case_when(
    salary > 70000 ~ "High",
    salary >= 40000 & salary <= 70000 ~ "Medium",
    TRUE ~ "Low"
  ))
```

Summary

- **Tidyverse provides concise, readable syntax**
- **PROC SQL is familiar to SAS users but more verbose**
- **Both methods offer powerful data manipulation tools**
- **Additional topics covered:** Case statements, multiple conditions, joins

Ready to dive deeper into Statistical Analysis?

Statistical Analysis: Basic Modeling & Hypothesis Testing

Introduction to Statistical Analysis

- Understanding key statistical concepts
- Comparing **SAS (PROC)** and **R (Base R & Tidyverse)**
- **Key Topics:**
  - Descriptive Statistics
  - Hypothesis Testing
  - Regression Analysis
  - Model Diagnostics

Descriptive Statistics

SAS (PROC MEANS & PROC FREQ)

```
PROC MEANS DATA=dataset MEAN MEDIAN STD MIN MAX;
   VAR age salary;
RUN;

PROC FREQ DATA=dataset;
   TABLES gender job_role;
RUN;
```

R (Base R & Tidyverse)

```
summary(dataset[, c("age", "salary")])

library(dplyr)
dataset %>% summarise(
  mean_age = mean(age, na.rm = TRUE),
  median_salary = median(salary, na.rm = TRUE),
  sd_salary = sd(salary, na.rm = TRUE)
)
```

Hypothesis Testing

t-Test (Comparing Means)

SAS

```
PROC TTEST DATA=mydata;
   CLASS group;
   VAR outcome;
RUN;
```

R

```
t.test(outcome ~ group, data = mydata)
```

z-Test (Comparing Proportions)

SAS

```
PROC FREQ DATA=mydata;
   TABLES group*outcome / CHISQ;
RUN;
```

Chi-Square Test (Categorical Variables)

SAS (PROC FREQ)

```
PROC FREQ DATA=dataset;
   TABLES gender*job_role / CHISQ;
RUN;
```

R (chisq.test Function)

```
table_data <- table(dataset$gender, dataset$job_role)
chisq.test(table_data)
```

Non-Parametric Tests

Wilcoxon Rank-Sum Test (Mann-Whitney U Test)

SAS

```
PROC NPAR1WAY WILCOXON DATA=mydata;
   CLASS group;
   VAR outcome;
RUN;
```

R

```
wilcox.test(outcome ~ group, data = mydata)
```

Kruskal-Wallis Test (Non-Parametric ANOVA)

SAS

```
PROC NPAR1WAY DATA=mydata WILCOXON;
   CLASS group;
   VAR outcome;
RUN;
```

Correlation Analysis

SAS

```
PROC CORR DATA=mydata;
   VAR var1 var2;
RUN;
```

R

```
cor.test(mydata$var1, mydata$var2)
```

ANOVA (Comparing Multiple Groups)

SAS (PROC ANOVA)

```
PROC ANOVA DATA=dataset;
    CLASS job_role;
    MODEL salary = job_role;
    MEANS job_role / TUKEY;
RUN;
```

R (aov Function)

```
anova_model <- aov(salary ~ job_role, data = dataset)
summary(anova_model)
```

Regression Analysis

Linear Regression

SAS (PROC REG)

```
PROC REG DATA=dataset;
   MODEL salary = age experience education;
RUN;
QUIT;
```

R (lm Function)

```
lm_model <- lm(salary ~ age + experience + education, data
summary(lm_model)
```

Logistic Regression

SAS (PROC LOGISTIC)

```
PROC LOGISTIC DATA=dataset;
   MODEL promotion (EVENT='1') = age experience salary;
RUN;
```

Model Diagnostics & Interpretation

SAS (PROC REG Diagnostics)

```
PROC REG DATA=dataset PLOTS=ALL;
   MODEL salary = age experience education;
RUN;
```

R (Diagnostic Plots)

```
par(mfrow=c(2,2))
plot(lm_model)
```

Summary

Descriptive statistics help understand data distribution.

Hypothesis tests (t-test, chi-square, ANOVA) determine significance.

Regression models (linear/logistic) predict outcomes.

Model diagnostics check the validity of results.

Ready to explore **Data Visualization**?

Visualization with R - ggplot2 vs. SAS

Introduction to Data Visualization

- Importance of effective data visualization
- Comparing **ggplot2 (R)** and **PROC SGPLOT (SAS)**
- Key visualization techniques:
  - **Bar Charts**
  - **Histograms & Density Plots**
  - **Scatter Plots**
  - **Box Plots**
  - **Facet Grids**
  - **Custom Themes & Labels**

Bar Charts

SAS (PROC SGPLOT)

```
PROC SGPLOT DATA=sashelp.class;
   VBAR age / RESPONSE=height STAT=MEAN;
RUN;
```

R (ggplot2)

```
library(ggplot2)
ggplot(data = class, aes(x = age, y = height)) +
  geom_bar(stat = "summary", fun = "mean")
```

Additional Features:

- **Stacked Bar Chart**: `geom_bar(position = "stack")`
- **Grouped Bar Chart**: `geom_bar(position = "dodge")`
- **Adding Labels**: `geom_text(aes(label = height))`

Histograms & Density Plots

SAS (PROC SGPLOT)

```
PROC SGPLOT DATA=sashelp.class;
   HISTOGRAM height / NBINS=10;
   DENSITY height;
RUN;
```

R (ggplot2)

```
ggplot(data = class, aes(x = height)) +
  geom_histogram(bins = 10, fill = "blue", alpha = 0.5) +
  geom_density(color = "red")
```

Enhancements:

- **Change bin width**: `geom_histogram(binwidth = 5)`
- **Overlay multiple distributions**

Scatter Plots

SAS (PROC SGPLOT)

```
PROC SGPLOT DATA=sashelp.class;
   SCATTER x=height y=weight;
RUN;
```

R (ggplot2)

```
ggplot(data = class, aes(x = height, y = weight)) +
  geom_point(color = "blue")
```

Additional Features:

- **Adding trend lines**: `geom_smooth(method = "lm")`
- **Coloring by category**: `aes(color = gender)`

Box Plots

SAS (PROC SGPLOT)

```
PROC SGPLOT DATA=sashelp.class;
   VBOX height / CATEGORY=age;
RUN;
```

R (ggplot2)

```
ggplot(data = class, aes(x = as.factor(age), y = height)) +
  geom_boxplot(fill = "lightblue")
```

Customizations:

- **Change colors**: `fill = "red"`
- **Add jittered points**: `geom_jitter()`

Facet Grids & Custom Themes

SAS (PROC SGPANEL)

```
PROC SGPANEL DATA=sashelp.class;
   PANELBY gender;
   SCATTER x=height y=weight;
RUN;
```

R (ggplot2)

```
ggplot(data = class, aes(x = height, y = weight)) +
  geom_point() +
  facet_wrap(~ gender)
```

Custom Themes:

```
ggplot(data = class, aes(x = height, y = weight)) +
  geom_point() +
  theme_minimal()
```

Summary

- **ggplot2 provides layered, flexible visualizations**
- **SAS PROC SGPLOT is powerful but less flexible**
- **Both support statistical overlays, facet grids, and custom themes**
- **Choosing the right tool depends on data complexity and output requirements**

Next, we'll dive into **SAS Macros vs. R Functions**

SAS Macros vs. R Functions

Introduction to Automation in SAS & R

- Why automate repetitive tasks?
- **SAS Macros** vs. **R Functions**
- Key comparisons:
  - **Concepts & Differences**
  - **Parameterization**
  - **Looping & Iteration**
  - **Conditional Execution**
  - **Modular Code Reuse**
  - **Debugging & Error Handling**

Concepts & Differences

Feature	SAS Macros	R Functions
Definition	Preprocessor for code automation	Self-contained reusable block of code
Execution	Executes before SAS compiles	Executes during runtime
Scope	Operates at the text level	Operates on objects/data
Debugging	Requires macro options (`MPRINT`, `SYMBOLGEN`)	Easier with built-in debugging tools
Flexibility	Primarily for SAS workflows	Used in multiple domains, including ML, visualization

Defining a Simple Macro/Function

SAS Macro

```
%MACRO summarize_data(dataset, var);
   PROC MEANS DATA=&dataset;
      VAR &var;
   RUN;
%MEND summarize_data;

%summarize_data(sashelp.class, height);
```

R Function

```
summarize_data <- function(dataset, var) {
  dataset %>%
    summarise(mean_value = mean(.data[[var]], na.rm = TRUE)
}

summarize_data(class, "height")
```

Looping & Iteration

SAS Macro Loop

```
%MACRO loop_example;
   %DO i = 1 %TO 5;
      %PUT Loop Iteration: &i;
   %END;
%MEND loop_example;

%loop_example;
```

R Function with Loop

```
loop_example <- function() {
  for (i in 1:5) {
    print(paste("Loop Iteration:", i))
  }
}

loop_example()
```

Conditional Execution in Clinical Analysis

SAS Macro for Adverse Event (AE) Count Check

```
%MACRO check_ae_count(dataset, threshold);
   PROC SQL NOPRINT;
      SELECT COUNT(*) INTO :ae_count
      FROM &dataset
      WHERE AE_SEVERITY = "Serious";
   QUIT;

   %IF &ae_count > &threshold %THEN %DO;
      %PUT High number of serious adverse events detected.;
   %END;
   %ELSE %DO;
      %PUT Serious adverse events are within acceptable lim
   %END;
%MEND check_ae_count;

%check_ae_count(adverse_events, 50);
```

Advanced Parameterization

SAS Macro with Multiple Parameters

```
%MACRO filter_data(dataset, column, value);
   PROC SQL;
      SELECT * FROM &dataset WHERE &column = "&value";
   QUIT;
%MEND filter_data;

%filter_data(sashelp.class, sex, "M");
```

R Function with Multiple Parameters

```
filter_data <- function(dataset, column, value) {
  dataset %>% filter(.data[[column]] == value)
}

filter_data(class, "sex", "M")
```

Debugging & Error Handling

SAS Macro Debugging

```
OPTIONS MPRINT SYMBOLGEN MLOGIC;
%filter_data(sashelp.class, age, 14);
```

R Function Debugging

```
debug(filter_data)
filter_data(class, "age", 14)
```

Comparison

Feature	SAS Macros	R Functions
Text Processing	Expands text-based macros	Works with objects and functions
Debugging Tools	`MPRINT`, `SYMBOLGEN`, `MLOGIC`	`debug()`, `traceback()`
Scope & Usage	SAS automation	General-purpose programming
Flexibility	Limited outside SAS	Usable across multiple domains
Data Processing	Used with `PROC` steps	Works within tidyverse

Summary

- **SAS Macros** automate repetitive tasks in SAS programs.
- **R Functions** provide a more flexible approach with better debugging.
- **Looping, conditional execution, and parameterization** are possible in both.
- **Choosing between them depends on the workflow and tool preference.**

Conclusion

Strengths of SAS in Clinical Trials

- **Regulatory Compliance**: Accepted by FDA, EMA, and PMDA
- **Data Handling**: Efficient with large clinical datasets (CDISC, SDTM, ADaM)
- **Validation & Standardization**: Pre-defined procedures ensure consistency
- **Industry Adoption**: Preferred by CROs and pharmaceutical companies
- **PROC Procedures**: Optimized for clinical trial data processing

Example: Generating Summary Statistics in SAS

```
PROC MEANS DATA=adam.adsl;
   VAR AGE HEIGHT WEIGHT;
   CLASS TRT;
   OUTPUT OUT=summary_stats MEAN= / AUTONAME;
RUN;
```

Strengths of R in Clinical Trials

- **Flexibility & Open Source**: Free and highly extensible with packages
- **Advanced Statistical Modeling**: Strong in ML, Bayesian methods, and visualization
- **Reproducibility**: R Markdown and Quarto facilitate transparent reporting
- **Interactive Data Exploration**: ggplot2, Shiny for dynamic visualization
- **Growing Regulatory Acceptance**: Used in exploratory and confirmatory analysis

Example: Generating Summary Statistics in R

```
summary_stats <- adsl %>%
  group_by(TRT) %>%
  summarise(across(c(AGE, HEIGHT, WEIGHT), mean, na.rm = TR
```

Key Differences: SAS vs. R

Feature	SAS	R
Regulatory Approval	Strong industry compliance	Increasing acceptance
Cost	Licensed software	Free & open-source
Ease of Use	Structured, PROC-based syntax	Flexible, function-based programming
Statistical Power	Robust for standard analyses	More advanced & customizable models
Graphics & Reporting	Basic visualization tools	Advanced (ggplot2, Shiny, Quarto)
Reproducibility	SAS programs, macros	R Markdown, Quarto, version control

Regulatory Considerations

- **SAS**: Pre-approved by agencies, easier for regulatory submissions
- **R**: Needs validation but is gaining traction (R-based RWE analysis, PK/PD modeling)
- **Hybrid Approach**: Many organizations use SAS for regulatory and R for exploratory analysis

Conclusion

- **SAS remains dominant for regulatory submissions** due to compliance and validation.
- **R excels in statistical modeling, visualization, and flexibility.**
- **Many organizations use both**: SAS for compliance, R for exploratory and advanced analytics.
- **Future Trend**: Increasing adoption of R in pharma, especially for real-world evidence (RWE) and adaptive trials.

Questions?

Thank You!